OFFICIAL TECHNICAL NOTES | VERSION 2.0 | INDIAN RAILWAYS IMS

TRAINSYNC — SYSTEM ARCHITECTURE & TECHNICAL DOCUMENTATION

Next-Generation Intelligent Railway Traffic Management, Dynamic Track Deviation, & Infrastructure Portal

Project Name	Dataset Origin	Network Scope	Live Deployment
TrainSync (Indian Railways IMS)	database.xlsx & MySQL Backend	Northern Railway (Delhi Division)	GitHub Pages Global Edge CDN

1. Executive Summary & Core Objective

TrainSync is an enterprise-grade railway operations and infrastructure management platform modeled after the real-world operational workflows of **Indian Railways**. It unifies network visualization, track health monitoring, live locomotive tracking, and autonomous conflict resolution into a unified, responsive interface.

The primary technical objective of TrainSync is **automated real-time track deviation**. In high-density railway networks, sudden blockages—such as flash flooding, structural track defects, sleeper replacements, or signal breakdowns—can cause cascading delays across the entire division. TrainSync solves this by coupling live telemetry with **Dijkstra's shortest path graph algorithm** to compute optimal detour routes and transmit instant reroute instructions to Station Masters and Loco Pilots.

2. Complete Technology Stack

TrainSync is constructed with a modern, high-performance web and data engineering stack:

Layer	Technology / Tool	Role & Implementation Architecture
Frontend Structure	Semantic HTML5	Document Object Model structure, canvas containers, navigation menus, accessible role views, and SVG icons.
Styling & Design System	Vanilla CSS3 (Design Tokens)	Dark-mode glassmorphism, responsive grid & flexbox layouts, GPU-accelerated keyframe animations, and custom scrollbars.
Typography	Google Fonts (Orbitron, Rajdhani, Inter)	Futuristic heads-up displays, railway station signage styling, and monospace data typography.

Layer	Technology / Tool	Role & Implementation Architecture
Geospatial Map Engine	HTML5 2D Canvas API	60FPS hardware-accelerated railway map, Mercator geographic projection, Bezier curves, animated dual rails, and station pins.
Routing Algorithm	Dijkstra's Algorithm (Priority Queue)	Bidirectional weighted graph traversal, real-time edge exclusion for blocked tracks, distance computation, and delay estimation.
State Management	Reactive App State + Web Storage API	Centralized `APP` state machine with persistent `localStorage` for work queries, maintenance jobs, and dispatched deviations.
Backend Engine	Python 3 + Flask	RESTful API endpoints (`/api/database` , `/api/stations` , `/api/tracks` , `/api/analyze` , `/api/construction`).
Database & Storage	MySQL 8.0 + OpenXML Parser	Relational 7-table schema with automated data ingestion from `database.xlsx` .
Deployment & CI/CD	Git + GitHub Pages Edge CDN	Global static hosting, automated Git deployments, zero-downtime branch publication.

3. System Architecture & Dual-Mode Pipeline

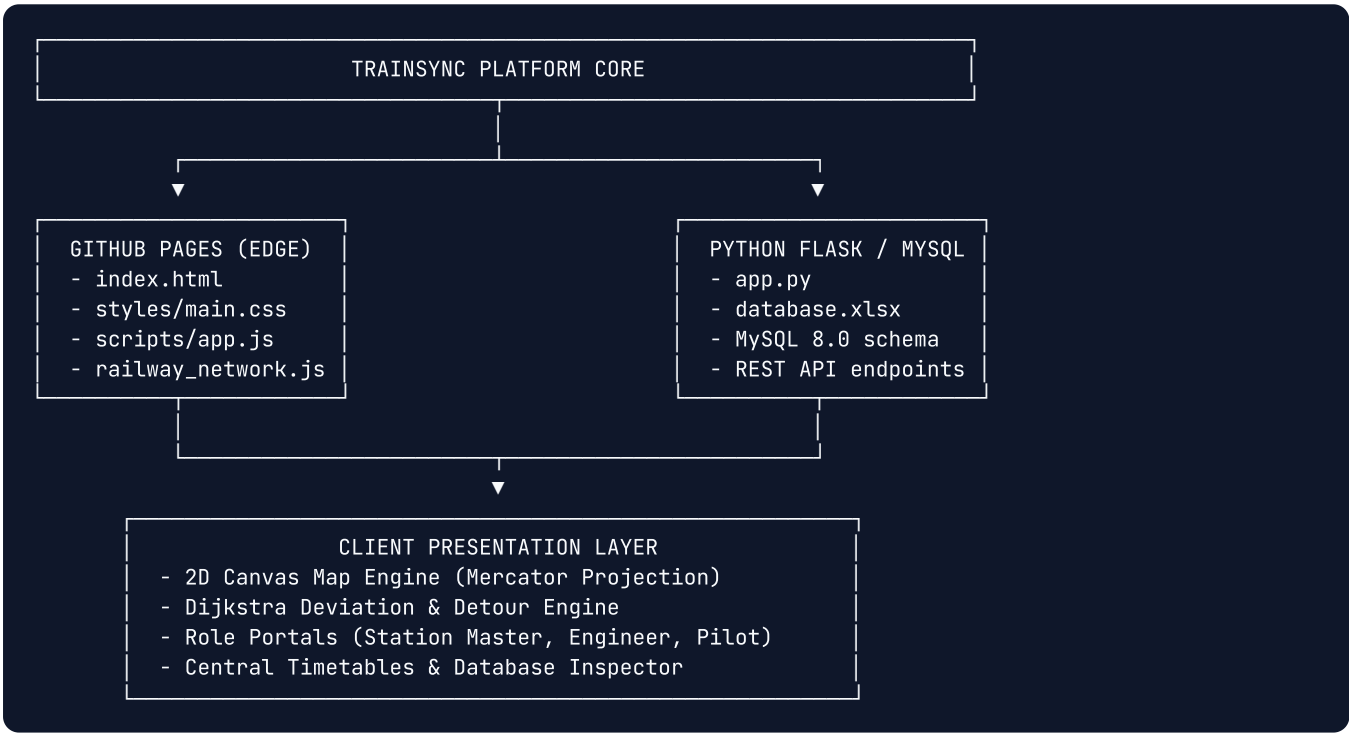
TrainSync implements a dual-mode deployment pipeline to support both edge environments and full server enterprise deployments:

🌐 Edge Pipeline (GitHub Pages)

GitHub Pages hosts static assets over a global CDN. The complete relational database parsed from `database.xlsx` is serialized into structured JSON inside `data/railway_network.js` and surfaced through a browser API mirror (`window.RailwayAPI`). Pathfinding and UI updates run client-side with 0ms server latency.

⚙️ Server Microservice (Python / MySQL)

For backend enterprise installations, `app.py` executes as a Flask microservice connected to MySQL. It parses `database.xlsx` , manages persistent SQL tables, and exposes REST endpoints for external systems.



4. Detailed Component Functionality

4.1. Animated Splash / Flash Screen

- **Particle Spark Canvas:** Simulates wheel-rail friction sparks using a custom particle physics vector engine with decay rates, random velocities, and radial alpha gradients.
- **Locomotive Vector Art:** A pure CSS-drawn steam engine and passenger coaches with glowing windows, animated wheels, and timed chimney steam puffs.
- **Staged System Initializer:** Simulates progressive authentication, database pre-loading, and route graph construction before launching the dashboard.

4.2. Geospatial Railway Map (Google Maps Style)

- **Mercator Geographic Projection:** Maps 60 stations across Northern Railway (Delhi Division) bounded between 28.1525°N – 28.8465°N and 76.7522°E – 77.8181°E:

```
x = pad_x + ((lon - min_lon) / (max_lon - min_lon)) * (width - 2 * pad_x)
y = pad_y + ((max_lat - lat) / (max_lat - min_lat)) * (height - 2 * pad_y)
```

- **Double-Rail Rendering:** Computes normal vectors perpendicular to track angles to render realistic parallel rails with sleeper ties.
- **Interactive Track Hitboxes:** Computes point-to-line segment distances to allow instant clicking and hovering over any track section for real-time status inspection.
- **Color-Coded Statuses:**
  - 🟢 **Available:** Emerald green double-rail with speed limit tags.
  - 🔴 **Maintenance / Closed:** Dashed red glowing lines with pulsing warning markers.
  - 🟡 **Busy:** Amber track indicating line congestion.
  - 🟠 **Active Deviation:** Neon orange route with animated direction chevrons.

4.3. Real-Time Autonomous Train Deviation System

When track maintenance or emergency closures occur (e.g. `TRK006` New Delhi → Delhi Cantt or `TRK009` Gurugram → Garhi Harsaru), the deviation system operates in five automated steps:

1. **Conflict Detection:** Scans active train route sequences against the `construction_blocks` table.
2. **Graph Reconstruction:** Generates an adjacency graph of available tracks, removing all blocked edges.
3. **Dijkstra Shortest Path Search:** Computes the minimal-distance path from the train's current position to its destination:

   ```
   dist[v] = min(dist[v], dist[u] + weight(u, v))
   ```

4. **Delay & Metrics Calculation:** Calculates excess distance in kilometers, estimated travel time based on track speed limits, and logs a formal `traffic_decision` record.
5. **Dual Broadcast Dispatch:**
   - **Station Master:** Previews the diversion path directly on the map, approves the decision, and transmits reroute telemetry.
   - **Loco Pilot:** The cab Heads-Up Display (HUD) immediately updates with the detour corridor, updated signals, and clearance flags.

4.4. Role-Based Specialized Portals

Portal / Role	Primary Tools & User Capabilities
Station Master	Division network map, track status toggling, emergency deviation modal, repair query creation, train scheduling oversight.
Repair Engineer	Construction block tracker, maintenance order management, damage diagnosis (rail fractures, flood subsidence, signaling defects).
Loco Pilot	Cab cockpit HUD, live speed telemetry, speed limit enforcement alerts, section clearance indicators, diversion notification panel.

5. Relational Database Schema (from database.xlsx)

The platform dataset represents the complete Northern Railway Delhi regional division:

- `stations` (60 records): `station_id`, `station_name`, `code`, `region`, `latitude`, `longitude`, `station_type`.
- `tracks` (34 records): `track_id`, `source_station_id`, `destination_station_id`, `distance_km`, `speed_limit`, `capacity`, `status`.
- `trains` (26 records): `train_id`, `train_number`, `train_name`, `train_type`, `priority`, `max_speed`, `length_m`.
- `schedules` (122 records): `schedule_id`, `train_id`, `station_id`, `arrival_time`, `departure_time`, `platform`.

- `routes` (96 records): `route_id`, `train_id`, `track_id`, `sequence_number`, `distance_km`.
- `construction_blocks` (5 active): `block_id`, `track_id`, `start_time`, `end_time`, `reason`, `priority`.
- `traffic_decisions` (8 decisions): `decision_id`, `train_number`, `decision`, `recommended_route`, `delay_minutes`.

6. Deployment Information & Quick Links

> **Live Website Deployment:**
> URL: `https://jiyag307-eng.github.io/main/`
> GitHub Repository: `https://github.com/jiyag307-eng/main` (Branch: `main`)
> PDF Notes File: `TrainSync_Documentation_and_Notes.pdf` in project root.